



Exercício 3.2: Três Services e a prova de DNS

Aluno: Luiz Brito · Data: 18/09/2026 · Disciplina: Cloud Native Development (CP1)

Ambiente

- Cluster local **kind** v0.32.0, Kubernetes **v1.36.1**, contexto `kind-fiap-store`, namespace `store-dev`.
- Pod `store-api` do Exercício 3.1 (`exercises/3.1/01-pod-store-api.yaml`) mais dois Pods, `store-api-2` e `store-api-3`, com o mesmo manifesto e os mesmos labels (`app=store-api`, `tier=backend`, `env=dev`), mudando só `metadata.name`. Resultado: 3 endpoints.
- A porta 30081 do NodePort está mapeada para o host em `assets/00-kind-cluster.yaml` (`extraPortMappings`), o que permite o acesso de fora do cluster no item (e).

Manifestos

03-svc-clusterip.yaml

```
apiVersion: v1
kind: Service
metadata:
  name: store-api
  namespace: store-dev
spec:
  type: ClusterIP
  sessionAffinity: ClientIP
  selector:
    app: store-api
  ports:
    - port: 80
      targetPort: http
```

O campo `sessionAffinity: ClientIP` só entra no item (g). Os itens (a) a (f) rodaram sem ele.

04-svc-nodeport.yaml

```
apiVersion: v1
kind: Service
metadata:
  name: store-api-np
  namespace: store-dev
spec:
  type: NodePort
  selector:
    app: store-api
  ports:
    - port: 80
      targetPort: http
      nodePort: 30081
```

05-svc-headless.yaml

```
apiVersion: v1
kind: Service
metadata:
  name: store-api-hl
  namespace: store-dev
spec:
  clusterIP: None
  selector:
    app: store-api
  ports:
    - port: 8000
      targetPort: http
```

Evidências

(a) Três Services e três endpoints em cada um

```
$ kubectl apply -f 03-svc-clusterip.yaml -f 04-svc-nodeport.yaml -f 05-svc-headless.yaml
service/store-api created
service/store-api-np created
service/store-api-hl created

$ kubectl get svc,endpoints -n store-dev
Warning: v1 Endpoints is deprecated in v1.33+; use discovery.k8s.io/v1 EndpointSlice
NAME                TYPE          CLUSTER-IP    EXTERNAL-IP    PORT(S)          AGE
service/store-api    ClusterIP     10.96.21.6    <none>         80/TCP           17s
service/store-api-hl ClusterIP     None          <none>         8000/TCP         17s
service/store-api-np NodePort      10.96.96.155  <none>         80:30081/TCP     17s

NAME                ENDPOINTS                                          AGE
endpoints/store-api  10.244.0.5:8000,10.244.0.6:8000,10.244.0.7:8000 17s
endpoints/store-api-hl 10.244.0.5:8000,10.244.0.6:8000,10.244.0.7:8000 17s
endpoints/store-api-np 10.244.0.5:8000,10.244.0.6:8000,10.244.0.7:8000 17s
```



Figura 1 (k9s): os três Pods `1/1 Running` com os IPs `10.244.0.5`, `.6` e `.7`.



Figura 2 (k9s): `store-api` (ClusterIP), `store-api-hl` (headless, sem IP) e `store-api-np` (NodePort 80:30081).

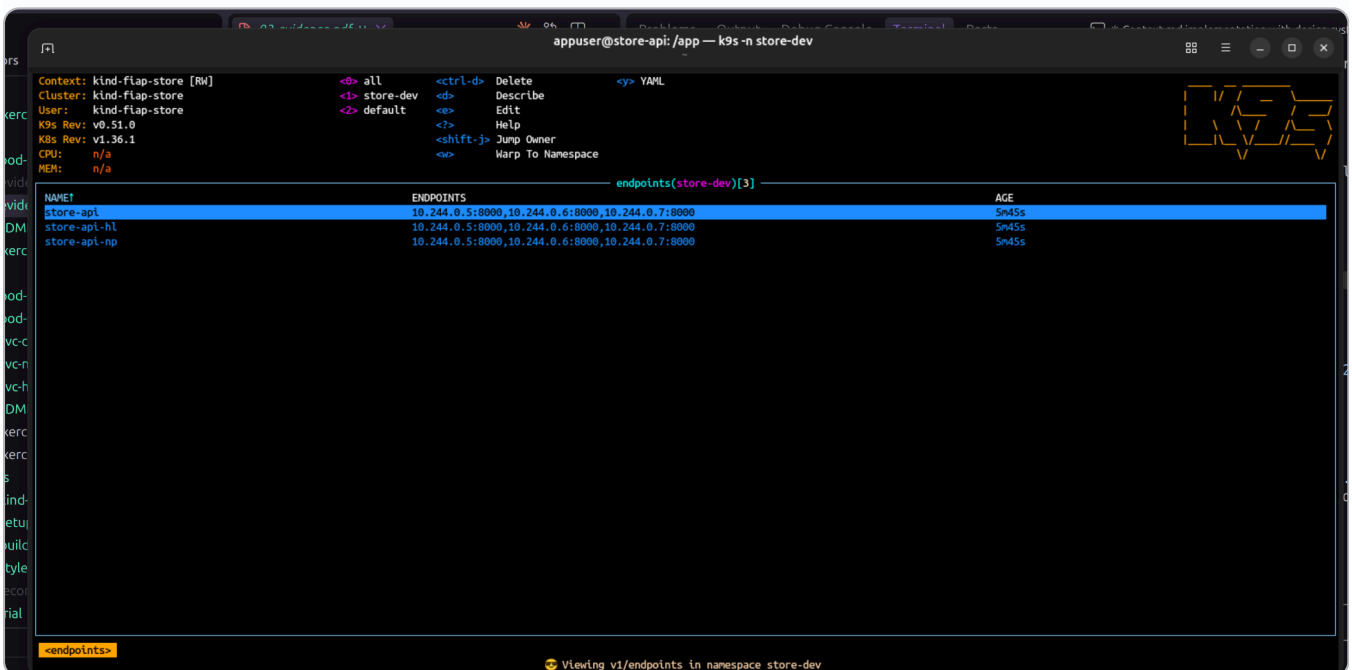


Figura 3 (k9s): os três Services com os mesmos 3 endpoints.

(b) EndpointSlice

```
$ kubectl get endpointslices -l kubernetes.io/service-name=store-api -n store-dev -o wide
```

NAME	ADDRESSTYPE	PORTS	ENDPOINTS	AGE
store-api-kw4pf	IPv4	8000	10.244.0.5,10.244.0.6,10.244.0.7	17s

Endpoints x EndpointSlice. `Endpoints` é um único objeto por Service com todos os endereços, reescrito por inteiro a cada mudança e limitado a 1000 endereços (está

depreciado desde o v1.33, como o aviso do item (a) mostra). `EndpointSlice` (`discovery.k8s.io/v1`) divide os endereços em fatias de até 100 endpoints, com condições por endpoint, e só a fatia alterada é reenviada, o que escala melhor.

(c) `nslookup` dos três Services

```
$ kubectl exec tester -n store-dev -- sh -c 'for s in store-api store-api-np store-api-hl; do echo "### $s"; nslookup $s.store-dev.svc.cluster.local | tail -n +4; done'
### store-api

Name:      store-api.store-dev.svc.cluster.local
Address: 10.96.21.6

### store-api-np
Name:      store-api-np.store-dev.svc.cluster.local
Address: 10.96.96.155

### store-api-hl
Name:      store-api-hl.store-dev.svc.cluster.local
Address: 10.244.0.5
Name:      store-api-hl.store-dev.svc.cluster.local
Address: 10.244.0.6
Name:      store-api-hl.store-dev.svc.cluster.local
Address: 10.244.0.7
```

O Pod `tester` usa a imagem `busybox:1.36`.

Por quê. ClusterIP e NodePort têm um IP virtual estável (`10.96.21.6` e `10.96.96.155`), e o DNS devolve só esse registro; quem distribui o tráfego entre os Pods é o kube-proxy. O Service headless (`clusterIP: None`) não tem IP virtual: o DNS devolve um registro `A` para cada Pod pronto (3 endereços) e o cliente fala direto com os Pods.

(d) 12 requisições ao ClusterIP, sem afinidade

```
$ kubectl exec curl-client -n store-dev -- sh -c 'for i in $(seq 12); do curl -s http://store-api/health; echo; done | sort | uniq -c'
  5 {"status":"ok","host":"store-api"}
  5 {"status":"ok","host":"store-api-2"}
  2 {"status":"ok","host":"store-api-3"}
```

O Pod `curl-client` usa a imagem `curlimages/curl:8.10.1`. As requisições se distribuem entre os 3 Pods, sem divisão exata: o kube-proxy escolhe o destino por conexão de forma aleatória, então 5/5/2 é um resultado normal.

(e) NodePort de fora do cluster

O `minikube service --url` não se aplica ao kind. A porta 30081 do nó está publicada no host (item Ambiente), então o acesso é direto:

```
$ curl -s localhost:30081/health
{"status":"ok","host":"store-api"}
```



Figura 4: `http://localhost:30081/health` no navegador respondendo por um dos Pods (`store-api-3`).

(f) Resolução entre namespaces

```
$ kubectl create namespace store-prod
namespace/store-prod created
$ kubectl run curl-prod --image=curlimages/curl:8.10.1 -n store-prod -- sleep 3600
pod/curl-prod created
$ kubectl exec curl-prod -n store-prod -- curl -sS -m 5 http://store-api/health
curl: (6) Could not resolve host: store-api
command terminated with exit code 6
$ kubectl exec curl-prod -n store-prod -- curl -sS -m 5 http://store-api.store-dev.svc.cluster.local/health
{"status":"ok","host":"store-api-2"}
$ kubectl exec curl-prod -n store-prod -- cat /etc/resolv.conf
search store-prod.svc.cluster.local svc.cluster.local cluster.local
nameserver 10.96.0.10
options ndots:5
$ kubectl exec curl-client -n store-dev -- cat /etc/resolv.conf
search store-dev.svc.cluster.local svc.cluster.local cluster.local
nameserver 10.96.0.10
options ndots:5
```

O papel do `search`. O nome curto `store-api` tem menos de 5 pontos (`ndots:5`), então o resolvedor tenta acrescentar cada domínio da lista `search`, em ordem: `store-api.store-prod.svc.cluster.local`, `store-api.svc.cluster.local` e `store-api.cluster.local`. Nenhum existe, porque o Service está em `store-dev`, e a resolução falha. Em `store-dev` a primeira tentativa (`store-api.store-dev.svc.cluster.local`) já acerta, por isso o nome curto só funciona dentro do mesmo namespace. Entre namespaces é preciso usar pelo menos `store-api.store-dev` ou o FQDN.

(g) `sessionAffinity: ClientIP`

```
$ diff -u 03-svc-clusterip.yaml.antes 03-svc-clusterip.yaml
@@ -5,6 +5,7 @@
     namespace: store-dev
   spec:
     type: ClusterIP
+   sessionAffinity: ClientIP
     selector:
       app: store-api
     ports:
```

```
$ kubectl apply -f 03-svc-clusterip.yaml
service/store-api configured

$ kubectl get svc store-api -n store-dev -o jsonpath='{.spec.sessionAffinity}{"\n"}'
ClientIP

$ kubectl exec curl-client -n store-dev -- sh -c 'for i in $(seq 12); do curl -s http://store-api/health; echo; done | sort | uniq -c'
12 {"status":"ok","host":"store-api"}
```

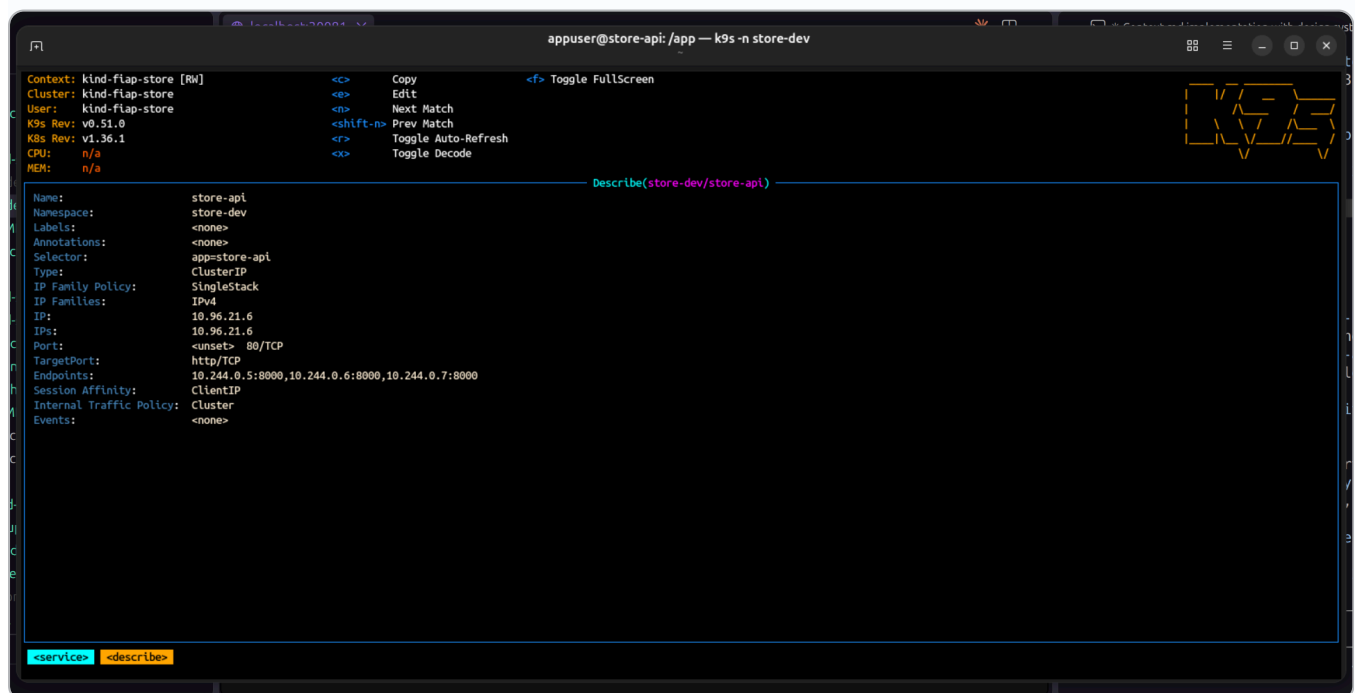


Figura 5 (`k9s, describe`): `Session Affinity: ClientIP` no Service `store-api`.

Com a afinidade por IP de origem, o kube-proxy manda todas as conexões do mesmo cliente para o mesmo Pod: as 12 requisições foram para `store-api`, contra 3 Pods diferentes no item (d).

Conclusão

Os sete itens foram cumpridos: os três Services listam 3 endpoints, o `nslookup` do headless devolve 3 endereços contra 1 dos demais, as 12 requisições atingiram 3 hostnames sem afinidade e exatamente 1 com `sessionAffinity: ClientIP`, o NodePort respondeu de fora do cluster, e o nome curto falhou entre namespaces enquanto o FQDN funcionou.